

CUDSS Raport 29.05.2026

Podobnie jak w poprzednim etapie prac, testy zostały przeprowadzone zarówno w środowisku lokalnym, jak i w środowisku chmurowym, co pozwoliło na wstępną ocenę zachowania biblioteki w różnych konfiguracjach sprzętowych.

W środowisku lokalnym wykorzystano komputer wyposażony w kartę graficzną NVIDIA GeForce RTX 3050 Laptop GPU posiadającą 4 GB pamięci VRAM oraz 16 GB pamięci RAM systemowej. Środowisko to wykorzystano głównie do weryfikacji poprawności implementacji, uruchamiania testów funkcjonalnych oraz przeprowadzania pomiarów dla mniejszych przypadków testowych.

Ze względu na ograniczenia pamięciowe środowiska lokalnego, większe przypadki testowe zostały uruchomione w środowisku chmurowym RunPod.io, gdzie wykorzystano serwer wyposażony w kartę NVIDIA RTX PRO 4500 dysponującą 24 GB pamięci VRAM, 62 GB pamięci RAM oraz 28 vCPU. Środowisko to umożliwiło przeprowadzenie bardziej wymagających testów wydajnościowych dla znacznie większych układów równań.

W analizach wykorzystano udostępnione przez opiekuna projektu macierze testowe. Ze względu na ograniczenia zasobów sprzętowych mniejsze instancje problemu analizowano w środowisku lokalnym, natomiast większe macierze, wymagające znacznie większej ilości pamięci operacyjnej i pamięci GPU, uruchamiano w środowisku chmurowym.

W celu zapewnienia przejrzystości prac oraz możliwości odtworzenia wyników, rozwijany kod źródłowy aplikacji został umieszczony w repozytorium GitHub (https://github.com/filipdukat/cudss_sparse_solver), co umożliwia bieżące śledzenie zmian, kontrolę wersji oraz łatwiejsze uruchamianie testów w różnych środowiskach obliczeniowych.

Celem przeprowadzonych prac było rozszerzenie środowiska testowego zgodnie z uwagami prowadzącego o mechanizmy pozwalające na ocenę dokładności numerycznej, czasu realizacji poszczególnych etapów obliczeń oraz zapotrzebowania pamięciowego dla różnych wariantów faktoryzacji.

Weryfikacja poprawności rozwiązania

Zgodnie z uwagami prowadzącego do środowiska testowego dodano mechanizm weryfikacji poprawności otrzymanego rozwiązania poprzez obliczanie błędu rezydualnego w postaci normy:

$$||Ax - b||$$

oraz błędu względnego:

$$\frac{||Ax-b||}{||b||}$$

Pozwala to na ocenę poprawności numerycznej rozwiązania. Dzięki temu możliwe jest wykrycie sytuacji, w których biblioteka zwraca rozwiązanie obciążone znacznym błędem numerycznym pomimo poprawnego zakończenia procedury obliczeniowej.

Przeprowadzone testy wykazały, że dla analizowanych przypadków wartości błędu względnego pozostają na poziomie rzędu 10^{-11} , co wskazuje na wysoką poprawność numeryczną uzyskiwanych rozwiązań.

Własności macierzy i dobór faktoryzacji

Zgodnie z uwagami prowadzącego przeanalizowano własności dostarczonych macierzy testowych. Są to macierze symetryczne, jednak nie dodatnio określone, co oznacza, że zastosowanie faktoryzacji Cholesky'ego (LL^T) nie jest właściwe. W przypadku biblioteki NVIDIA cuDSS zaimplementowano dwa tryby pracy solvera:

- General LU — traktujący macierz jako ogólną, bez wykorzystania informacji o symetrii,
- Symmetric LDL^T — wykorzystujący fakt, że macierz jest symetryczna.

Pozwoliło to przeprowadzić bezpośrednie porównanie obu wariantów zgodnie z sugestią prowadzącego.

Wyniki testów jednoznacznie wskazują, że wykorzystanie struktury symetrycznej macierzy znacząco poprawia efektywność obliczeń. Dla mniejszego przypadku testowego (472096 równań) uzyskano następujące wyniki:

Parametr	LU	LDL ^T
Relative residual	8.957467209477e-12	8.768980861648e-12
Analysis time	3203.930 ms	3000.237 ms
Factorization time	2329.570 ms	1694.495 ms
Solve time	51.871 ms	37.615 ms
Memory after analysis	656.000 MB	392.000 MB
Memory after factorization	2598.000 MB	1426.000 MB
Memory after solve	2598.000 MB	1426.000 MB
Estimated factor storage	1942.000 MB	1034.000 MB
Peak GPU memory	2598.000 MB	1426.000 MB

```

=====
cuDSS Sparse Solver
=====
Loading COO matrix...
Converting COO -> CSR...
Loading RHS vector...
Vector size: 472096
isComplex: 0
nonzero b[60] = 1.4037231402
nonzero b[61] = 1.2264463014
nonzero b[228] = -1.3456898137
nonzero b[229] = -1.2555013257
nonzero b[902] = 1.7451716045
nonzero b[903] = 1.1732871696
nonzero b[904] = 1.8387848537
nonzero b[905] = 1.2509337720
nonzero b[906] = 1.4916580126
nonzero b[907] = 1.1390782477
Total nonzero b values: 437
Choose solver mode:
1 - General LU
2 - Symmetric LDL^T
Selection: 1
Selected mode: General LU
Rows: 472096
NNZ : 17592472
Allocating GPU memory...
Copying data to GPU...
Running analysis...
Running factorization...
Running solve...
=====
SOLVE FINISHED
=====

```

Rys. 1. Wyniki testów wydajnościowych dla lokalnego wykonania obliczeń dla macierzy o 472096 wierszach z wykorzystaniem faktoryzacji LU. 1/2

```

=====
SOLVE FINISHED
=====
First 10 x values:
x[0] = 0.0243214209
x[1] = -0.0010710765
x[2] = 0.0009010353
x[3] = 0.0025117503
x[4] = 0.0043860996
x[5] = -0.0026682566
x[6] = 0.0023424761
x[7] = 0.0032004068
x[8] = 0.0616945124
x[9] = 0.0794911330
=====
RESIDUAL CHECK
=====
||Ax - b|| = 1.255545151653e-10
||b|| = 1.401674292846e+01
relative residual = 8.957467209477e-12
=====
PERFORMANCE TESTS RESULTS
=====
Analysis time           : 3203.930 ms
Factorization time      : 2329.570 ms
Solve time              : 51.871 ms
Memory after analysis    : 656.000 MB
Memory after factorization: 2598.000 MB
Memory after solve       : 2598.000 MB
Estimated factor storage : 1942.000 MB
Peak GPU memory         : 2598.000 MB

```

Rys. 2. Wyniki testów wydajnościowych dla lokalnego wykonania obliczeń dla macierzy o 472096 wierszach z wykorzystaniem faktoryzacji LU. 2/2

```

=====
cuDSS Sparse Solver
=====
Loading COO matrix...
Converting COO -> CSR...
Loading RHS vector...
Vector size: 472096
isComplex: 0
nonzero b[60] = 1.4037231402
nonzero b[61] = 1.2264463014
nonzero b[228] = -1.3456898137
nonzero b[229] = -1.2555013257
nonzero b[902] = 1.7451716045
nonzero b[903] = 1.1732871696
nonzero b[904] = 1.8387848537
nonzero b[905] = 1.2509337720
nonzero b[906] = 1.4916580126
nonzero b[907] = 1.1390782477
Total nonzero b values: 437
Choose solver mode:
1 - General LU
2 - Symmetric LDL^T
Selection: 2
Selected mode: Symmetric LDL^T
Rows: 472096
NNZ : 9032284
Allocating GPU memory...
Copying data to GPU...
Running analysis...
Running factorization...
Running solve...
=====
SOLVE FINISHED
=====

```

Rys. 3. Wyniki testów wydajnościowych dla lokalnego wykonania obliczeń dla macierzy o 472096 wierszach z wykorzystaniem faktoryzacji LDL^T 1/2.

```

=====
SOLVE FINISHED
=====
First 10 x values:
x[0] = 0.0243214209
x[1] = -0.0010710765
x[2] = 0.0009010353
x[3] = 0.0025117503
x[4] = 0.0043860996
x[5] = -0.0026682566
x[6] = 0.0023424761
x[7] = 0.0032004068
x[8] = 0.0616945124
x[9] = 0.0794911330
=====
RESIDUAL CHECK
=====
||Ax - b|| = 1.229125504823e-10
||b|| = 1.401674292846e+01
relative residual = 8.768980861648e-12
=====
PERFORMANCE TESTS RESULTS
=====
Analysis time           : 3000.237 ms
Factorization time      : 1694.495 ms
Solve time              : 37.615 ms
Memory after analysis    : 392.000 MB
Memory after factorization: 1426.000 MB
Memory after solve       : 1426.000 MB
Estimated factor storage : 1034.000 MB
Peak GPU memory         : 1426.000 MB

```

Rys. 4. Wyniki testów wydajnościowych dla lokalnego wykonania obliczeń dla macierzy o 472096 wierszach z wykorzystaniem faktoryzacji LDL^T 2/2.

Dla większego przypadku testowego (1432119 równań) różnice były jeszcze bardziej widoczne:

Parametr	LU	LDLT ^T
Relative residual	1.197300676619e-11	1.107006087752e-11
Analysis time	5423.964 ms	5354.912 ms
Factorization time	13214.991 ms	2159.504 ms
Solve time	6038.772 ms	66.047 ms
Memory after analysis	3656.000 MB	2042.000 MB
Memory after factorization	13530.000 MB	7560.000 MB
Memory after solve	13530.000 MB	7560.000 MB
Estimated factor storage	9874.000 MB	5518.000 MB
Peak GPU memory	13530.000 MB	7560.000 MB

```

=====
cuDSS Sparse Solver
=====
Loading COO matrix...
Converting COO -> CSR...
Loading RHS vector...
Vector size: 1432119
isComplex: 0
nonzero b[230] = 1.4250954353
nonzero b[231] = 1.3015408949
nonzero b[295] = -1.2995078180
nonzero b[296] = -1.2035596094
nonzero b[5597] = 1.5647116548
nonzero b[5598] = 1.1981966031
nonzero b[5600] = 1.7843628104
nonzero b[5601] = 1.0697386599
nonzero b[5602] = 1.7768972228
nonzero b[5603] = 1.5555937827
Total nonzero b values: 1099
Choose solver mode:
1 - General LU
2 - Symmetric LDL^T
Selection: 1
Selected mode: General LU
Rows: 1432119
NNZ : 107043349
Allocating GPU memory...
Copying data to GPU...
Running analysis...
Running factorization...
Running solve...
=====
SOLVE FINISHED
=====

```

Rys. 5. Wyniki testów wydajnościowych dla wykonania obliczeń w środowisku chmurowym dla macierzy o 1432119 wierszach z wykorzystaniem faktoryzacji LU 1/2.

```

=====
SOLVE FINISHED
=====
First 10 x values:
x[0] = -0.0382466308
x[1] = -0.0052125733
x[2] = -0.0126186966
x[3] = -0.0877277121
x[4] = -0.0988278185
x[5] = -0.0621376332
x[6] = -0.0841807069
x[7] = -0.0871313687
x[8] = -0.0844951717
x[9] = -0.1171131090
=====
RESIDUAL CHECK
=====
||Ax - b|| = 1.853839423170e-10
||b|| = 1.548349098412e+01
relative residual = 1.197300676619e-11
=====
PERFORMANCE TESTS RESULTS
=====
Analysis time           : 5423.964 ms
Factorization time      : 13214.991 ms
Solve time              : 6038.772 ms
Memory after analysis   : 3656.000 MB
Memory after factorization: 13530.000 MB
Memory after solve      : 13530.000 MB
Estimated factor storage : 9874.000 MB
Peak GPU memory         : 13530.000 MB

```

Rys. 6. Wyniki testów wydajnościowych dla wykonania obliczeń w środowisku chmurowym dla macierzy o 1432119 wierszach z wykorzystaniem faktoryzacji LU 2/2.

```

=====
cuDSS Sparse Solver
=====
Loading COO matrix...
Converting COO -> CSR...
Loading RHS vector...
Vector size: 1432119
isComplex: 0
nonzero b[230] = 1.4250954353
nonzero b[231] = 1.3015408949
nonzero b[295] = -1.2995078180
nonzero b[296] = -1.2035596094
nonzero b[5597] = 1.5647116548
nonzero b[5598] = 1.1981966031
nonzero b[5600] = 1.7843628104
nonzero b[5601] = 1.0697386599
nonzero b[5602] = 1.7768972228
nonzero b[5603] = 1.5555937827
Total nonzero b values: 1099
Choose solver mode:
1 - General LU
2 - Symmetric LDL^T
Selection: 2
Selected mode: Symmetric LDL^T
Rows: 1432119
NNZ : 54237734
Allocating GPU memory...
Copying data to GPU...
Running analysis...
Running factorization...
Running solve...
=====
SOLVE FINISHED
=====

```

Rys. 7. Wyniki testów wydajnościowych dla wykonania obliczeń w środowisku chmurowym dla macierzy o 1432119 wierszach z wykorzystaniem faktoryzacji LDL^T 1/2.

```

=====
SOLVE FINISHED
=====
First 10 x values:
x[0] = -0.0382466308
x[1] = -0.0052125733
x[2] = -0.0126186966
x[3] = -0.0877277121
x[4] = -0.0988278185
x[5] = -0.0621376332
x[6] = -0.0841807069
x[7] = -0.0871313687
x[8] = -0.0844951717
x[9] = -0.1171131090
=====
RESIDUAL CHECK
=====
||Ax - b|| = 1.714031877906e-10
||b|| = 1.548349098412e+01
relative residual = 1.107006087752e-11
=====
PERFORMANCE TESTS RESULTS
=====
Analysis time           : 5354.912 ms
Factorization time      : 2159.504 ms
Solve time              : 66.047 ms
Memory after analysis   : 2042.000 MB
Memory after factorization: 7560.000 MB
Memory after solve      : 7560.000 MB
Estimated factor storage : 5518.000 MB
Peak GPU memory         : 7560.000 MB

```

Rys. 7. Wyniki testów wydajnościowych dla wykonania obliczeń w środowisku chmurowym dla macierzy o 1432119 wierszach z wykorzystaniem faktoryzacji LDL^T 2/2.

Uzyskane wyniki potwierdzają uwagę opiekuna - wykorzystanie faktoryzacji LDL^T dla symetrycznych macierzy niedodatnio określonych jest zdecydowanie bardziej efektywne niż stosowanie ogólnej faktoryzacji LU. Oba warianty zapewniają porównywalną dokładność numeryczną, jednak LDL^T znacząco redukuje czas faktoryzacji oraz zapotrzebowanie na pamięć GPU.

Monitorowanie czasu realizacji i zużycia pamięci

Zgodnie z sugestiami prowadzącego rozszerzono środowisko testowe o szczegółowe monitorowanie parametrów wydajnościowych. Obecnie dla każdej próby rejestrowane są:

- czas analizy strukturalnej (analysis time),
- czas faktoryzacji numerycznej (factorization time),
- czas rozwiązywania układu (solve time),
- zużycie pamięci po etapie analizy,
- zużycie pamięci po etapie faktoryzacji,
- zużycie pamięci po zakończeniu obliczeń,
- estymowana pamięć zajmowana przez przechowywanie faktorów,
- szczytowe zużycie pamięci GPU (peak memory).

Monitorowanie pamięci realizowane jest poprzez zewnętrzny pomiar dostępnej pamięci GPU z wykorzystaniem funkcji **cudaMemGetInfo()**, co pozwala na praktyczną ocenę rzeczywistego zapotrzebowania pamięciowego solvera.

Uzyskane dane umożliwiają bezpośrednie porównywanie bibliotek nie tylko pod względem poprawności obliczeń, ale również ich efektywności czasowej i pamięciowej, co będzie kluczowe przy dalszej analizie przydatności poszczególnych rozwiązań.

Funkcjonalność bibliotek i warianty użycia

Biblioteka cuDSS rozdziela proces rozwiązywania układu równań na trzy etapy: analysis, factorization oraz solve. Etap analysis odpowiada analizie struktury macierzy, planowaniu alokacji pamięci oraz przygotowaniu metadanych solvera, natomiast etap factorization wykonuje właściwą faktoryzację numeryczną (w naszym przypadku LU lub LDL^T), a etap solve realizuje podstawienie w przód/wstecz. Taki podział pozwala na ponowne wykorzystanie przygotowanej faktoryzacji przy rozwiązywaniu wielu układów z tą samą macierzą, ale różnymi prawymi stronami.

cuDSS wspiera rozwiązywanie układów z wieloma prawymi stronami (multiple RHS). Wynika to z interfejsu API, gdzie prawa strona i rozwiązanie tworzone są jako macierze gęste z parametrem nrhs, co pozwala rozwiązywać problem w postaci

$$AX = B,$$

a nie tylko pojedynczy układ

$$Ax = b$$

Oznacza to, że jedna analiza i jedna faktoryzacja mogą zostać wykorzystane do efektywnego rozwiązania wielu układów, co znacząco poprawia wydajność w odpowiednich scenariuszach.

W przypadku rzadkiej prawej strony (sparse RHS) biblioteka cuDSS w obecnym API nie oferuje natywnego wsparcia. Prawa strona tworzona jest jako macierz gęsta (cuDssMatrixCreateDn), nawet jeśli większość jej elementów jest zerowa. W naszym przypadku jest to szczególnie istotne, ponieważ testowany wektor prawej strony zawiera jedynie 437 niezerowych elementów przy 472 096 wierszach, co oznacza bardzo wysoką rzadkość. W praktyce oznacza to, że cuDSS przechowuje taki wektor w formacie gęstym, co może prowadzić do niepotrzebnego zużycia pamięci i być potencjalną wadą względem bibliotek wspierających sparse RHS.

Jeśli chodzi o obsługiwane formaty macierzy, cuDSS nie ogranicza się wyłącznie do CSR. W naszych testach wykorzystujemy format CSR, ponieważ wejściowe dane są ładowane jako COO i następnie konwertowane do CSR przed przekazaniem do solvera. Biblioteka wspiera również inne reprezentacje macierzy rzadkich np. COO (Coordinate Format) oraz CSC (Compressed Sparse Column), a także operacje na macierzach gęstych.

cuDSS pozwala również na monitorowanie czasu realizacji poszczególnych etapów obliczeń, co zostało już zaimplementowane. Oddzielnie mierzone są czasy dla etapu analizy, faktoryzacji i rozwiązywania układu, co umożliwia dokładniejsze porównywanie solverów oraz identyfikację etapów dominujących koszt obliczeń.

W zakresie monitorowania zużycia pamięci biblioteka nie udostępnia bezpośredniego mechanizmu raportowania szczegółowych statystyk pamięci, jednak możliwe jest zewnętrzne monitorowanie wykorzystania pamięci GPU przy użyciu cudaMemGetInfo(). W aktualnym benchmarku mierzone są: zużycie pamięci po analizie, po faktoryzacji, po etapie solve, szczytowe zużycie pamięci (peak memory) oraz estymowana pamięć zajmowana przez faktory, wyznaczana jako różnica pomiędzy zużyciem pamięci po faktoryzacji i po analizie.

Dodatkowo testy wykazały, że cuDSS poprawnie wspiera zarówno tryb ogólny (LU), jak i tryb wykorzystujący symetrię macierzy (LDL^T). W przypadku naszych symetrycznych macierzy wariant LDL^T osiągnął porównywalną dokładność numeryczną, przy

jednoczesnym znacznym zmniejszeniu zużycia pamięci GPU oraz skróceniu czasu faktoryzacji względem wariantu LU.

Dodatkowe uwagi

W kontekście stabilności pomiarów przeanalizowano również zagadnienie migracji wątków pomiędzy rdzeniami procesora. W przypadku środowisk CPU możliwe jest ograniczenie tego zjawiska poprzez mechanizmy przypisania procesu lub wątków do konkretnych rdzeni (CPU affinity / thread pinning), np. z wykorzystaniem `SetProcessAffinityMask` w środowisku Windows lub narzędzi `taskset` / `numactl` w systemach Linux.

W przypadku analizowanej biblioteki NVIDIA cuDSS wpływ tego zjawiska jest potencjalnie mniejszy niż dla solverów stricte CPU, ponieważ główne etapy obliczeniowe (analysis, factorization, solve) realizowane są przede wszystkim na GPU. Aspekt ten został uwzględniony i będzie brany pod uwagę przy późniejszych testach porównawczych, szczególnie w przypadku bibliotek wykorzystujących intensywnie zasoby CPU.